



**NANYANG
TECHNOLOGICAL
UNIVERSITY**
SINGAPORE

SC3020 Database System Principles

Group 8

Name	Matriculation Number	Individual Contributions
BHATI NANCY	U2222368G	20%
BRIAN TEY ZHI FONG	U2221714E	20%
BUI KHANH HUNG	U2322190J	20%
NYIM BOEY	U2320429D	20%
SAM CHERN KAI, CHESTER	U2322722J	20%

Table of Contents

Query Optimization.....	3
Task 1:.....	3
Task 2:.....	5
Task 3:.....	7
Task 4:.....	8
Task 5:.....	10
Task 6:.....	12
Task 7:.....	14
Task 8:.....	17
Task 9:.....	18
Task 10:.....	20
Transactions.....	23
Task 11:.....	23
Task 12:.....	24
Task 13:.....	26

Query Optimization

1. *explain select * from lineitem;*

Returns:

	QUERY PLAN text	
1	Seq Scan on lineitem (cost=0.00..172607.84 rows=6000784 width=117)	

2. *explain analyze select * from lineitem;*

Returns:

	QUERY PLAN text	
1	Seq Scan on lineitem (cost=0.00..172607.84 rows=6000784 width=117) (actual time=0.544..928.641 rows=6001215.00 loops=1)	
2	Buffers: shared hit=8942 read=103658	
3	Planning Time: 0.106 ms	
4	Execution Time: 1414.779 ms	

Task 1:

Find out the syntax of d+. Identify the index PostgreSQL creates on each table automatically.

Answer:

- d+ command syntax: \d+ [table name]
- PostgreSQL creates **B+ tree index** on **Primary Keys** on default

Table	Index
customer	c_custkey
lineitem	l_orderkey, l_partkey, l_suppkey, l_linenum
nation	n_nationkey
orders	o_orderkey
part	p_partkey
partsupp	ps_partkey, ps_suppkey

region	r_regionkey
supplier	s_suppkey

```
SC3020 Project 2=# \d+ region
Table "public.region"
  Column      |          Type          | Collation | Nullable | Default | Storage  | Compression | Stats target | Descri
ption
-----+-----+-----+-----+-----+-----+-----+-----+-----
 r_regionkey  | integer                |           | not null |         | plain    |             |             |
 r_name       | character(25)          |           | not null |         | extended |             |             |
 r_comment    | character varying(152) |           |         |         | extended |             |             |
Indexes:
    "region_pkey" PRIMARY KEY, btree (r_regionkey)
Referenced by:
    TABLE "nation" CONSTRAINT "fk_nation" FOREIGN KEY (n_regionkey) REFERENCES region(r_regionkey)
Not-null constraints:
    "region_r_regionkey_not_null" NOT NULL "r_regionkey"
    "region_r_name_not_null" NOT NULL "r_name"
Access method: heap
```

Task 2:

- (i) Explain in plain words the query plan that PostgreSQL selects to answer the query.
- (ii) Run the query for a second time. Does it “feel” faster? How much faster? Why did it happen?

Answer:

- (i) The query first does a sequential scan of every row in the lineitem table. Then, for each row, it checks whether `l_quantity >= 50` is true and removes the rows that do not hold true.
 - As there was no index that exists on `l_quantity`, the optimizer determined that scanning the entire table is the most efficient method to find matching rows.

1st Run:

	QUERY PLAN text	
1	Gather (cost=1000.00..156698.73 rows=118424 width=117) (actual time=1.458..1000.601 rows=119846.00 loops=1)	
2	Workers Planned: 2	
3	Workers Launched: 2	
4	Buffers: shared hit=95 read=112505	
5	-> Parallel Seq Scan on lineitem (cost=0.00..143856.33 rows=49343 width=117) (actual time=1.175..961.704 rows=39948.67 loo...	
6	Filter: (l_quantity >= '50'::numeric)	
7	Rows Removed by Filter: 1960456	
8	Buffers: shared hit=95 read=112505	
9	Planning:	
10	Buffers: shared hit=72 read=4 dirtied=1	
11	Planning Time: 5.785 ms	
12	Execution Time: 1005.577 ms	

(ii) The second query executed much faster (about 670ms faster) due to caching. This can be seen via higher shared hits in buffers in run 2 compared to run 1.

- It reused cached pages in memory which avoided slow disk reads and sped up execution time by 3 times.

2nd Run:

	QUERY PLAN	
	text	
1	Gather (cost=1000.00..156698.73 rows=118424 width=117) (actual time=0.645..322.230 rows=119846.00 loops=1)	
2	Workers Planned: 2	
3	Workers Launched: 2	
4	Buffers: shared hit=378 read=112222	
5	-> Parallel Seq Scan on lineitem (cost=0.00..143856.33 rows=49343 width=117) (actual time=0.317..284.397 rows=39948.67 loo...	
6	Filter: (l_quantity >= '50'::numeric)	
7	Rows Removed by Filter: 1960456	
8	Buffers: shared hit=378 read=112222	
9	Planning Time: 0.116 ms	
10	Execution Time: 327.083 ms	

Task 3:

- (i) Explain in plain words the query plan that PostgreSQL selects to answer the query.
- (ii) Is it different from the plan chosen in Task 2? Why?

Answer:

(i) PostgreSQL first uses the B+ tree index to find pages with `l_quantity >= 50`. For this query, it utilised bitmap scan, which is a combination of bitmap heap scan and bitmap index scan. The bitmap index scan first returns a bitmap which is a list of matching page IDs. Bitmap heap scan then reads the bitmap for those table pages, rechecks the condition and fetches tuples that fulfills the condition. The results then are combined into one final output and displayed.

(ii) It is different from Task 2 as there is an index on `l_quantity`. Bitmap scan is used because there is a moderate amount of data to be fetched, and it helps to reduce I/O cost by making random disk access into sequential access based on the optimizer.

```
Indexes:
  "lineitem_pkey" PRIMARY KEY, btree (l_orderkey, l_partkey, l_suppkey, l_linenumbe...
  "idx_lineitem_quantity" btree (l_quantity)
```

	QUERY PLAN
	text
1	Gather (cost=3222.22..150526.71 rows=118424 width=117) (actual time=28.436..251.577 rows=119846.00 loops=1)
2	Workers Planned: 2
3	Workers Launched: 2
4	Buffers: shared hit=40 read=74375 written=12
5	-> Parallel Bitmap Heap Scan on lineitem (cost=2222.22..137684.31 rows=49343 width=117) (actual time=9.729..204.208 rows=39948.67 loops=1)
6	Recheck Cond: (l_quantity >= '50'::numeric)
7	Rows Removed by Index Recheck: 571452
8	Heap Blocks: exact=12525 lossy=9980
9	Buffers: shared hit=40 read=74375 written=12
10	Worker 0: Heap Blocks: exact=14622 lossy=11950
11	Worker 1: Heap Blocks: exact=13739 lossy=11229
12	-> Bitmap Index Scan on idx_lineitem_quantity (cost=0.00..2192.61 rows=118424 width=0) (actual time=20.907..20.908 rows=119846.00 loops=1)
13	Index Cond: (l_quantity >= '50'::numeric)
14	Index Searches: 1
15	Buffers: shared read=330
16	Planning:
17	Buffers: shared hit=20 read=2
18	Planning Time: 2.093 ms
19	Execution Time: 256.882 ms

Task 4:

(i) Explain in plain words the query plan that PostgreSQL selects to answer the query. Is it different from the plan chosen in Task 3? Why?

(ii) Explain in plain words the query plan that PostgreSQL selects to answer the query. Is it different from the plan chosen for the previous query? Why?

Answer:

(i) In task 3, PostgreSQL used a B-tree index that can handle range predicates. In this query, PostgreSQL is executing a parallel sequential scan on the lineitem table, even though a hash index has been created on the l_quantity attribute. This is because a hash index supports equality comparisons (=) and cannot be used for range conditions such as >=. As a result, all the tuples are being scanned in parallel using two worker processes and applies the filter condition l_quantity >=50 to each row which allows efficient parallel processing but still requires reading the entire table.

QUERY PLAN	
text	
1	Gather (cost=1000.00..156658.73 rows=118024 width=117) (actual time=3.911..1884.903 rows=119846.00 loops=1)
2	Workers Planned: 2
3	Workers Launched: 2
4	Buffers: shared hit=94 read=112506
5	-> Parallel Seq Scan on lineitem (cost=0.00..143856.33 rows=49177 width=117) (actual time=1.690..1606.005 rows=39948.67 loo...
6	Filter: (l_quantity >= '50'::numeric)
7	Rows Removed by Filter: 1960456
8	Buffers: shared hit=94 read=112506
9	Planning:
10	Buffers: shared hit=16 read=1 dirtied=2
11	Planning Time: 2.764 ms
12	Execution Time: 1891.922 ms

(ii) In task 3, PostgreSQL uses hash index on l_quantity since the condition involves equality (=) which is compatible with hash indexing. The optimiser chooses a bitmap index scan followed by bitmap heap scan. The bitmap index scan first identifies the pages that may contain matching tuples, while the bitmap heap scan reads those pages, rechecks the condition and retrieves the qualifying rows.

In comparison with the previous query, the execution time is longer for this query. The possible reason could be a large number of tuples satisfying the condition and additional rechecks.

	QUERY PLAN	
	text	
1	Gather (cost=4686.69..151875.80 rows=118024 width=117) (actual time=27.901..4322.434 rows=119846.00 loops=1)	
2	Workers Planned: 2	
3	Workers Launched: 2	
4	Buffers: shared hit=333 read=74048	
5	-> Parallel Bitmap Heap Scan on lineitem (cost=3686.69..139073.40 rows=49177 width=117) (actual time=15.505..4100.128 rows=39948.67 loops=3)	
6	Recheck Cond: (l_quantity = '50'::numeric)	
7	Rows Removed by Index Recheck: 571452	
8	Heap Blocks: exact=14476 lossy=11947	
9	Buffers: shared hit=333 read=74048	
10	Worker 0: Heap Blocks: exact=13409 lossy=10926	
11	Worker 1: Heap Blocks: exact=13001 lossy=10286	
12	-> Bitmap Index Scan on idx_lineitem_quantity_hash (cost=0.00..3657.18 rows=118024 width=0) (actual time=20.180..20.181 rows=119846.00 loo...	
13	Index Cond: (l_quantity = '50'::numeric)	
14	Index Searches: 1	
15	Buffers: shared hit=296	
16	Planning:	
17	Buffers: shared hit=4 read=2	
18	Planning Time: 2.658 ms	
19	Execution Time: 4330.166 ms	

Task 5:

- (i) Explain in plain words the query plan that PostgreSQL selects to answer the query.
- (ii) Run the same query again and explain in plain words the query plan that PostgreSQL selects to answer the query. Is it different from the plan used for 1MB of working memory? Why?

Answer:

- (i) `set work_mem = '1MB';`

With limited memory, PostgreSQL performs Parallel Sequential Scan on lineitem table (line 17) followed by a GroupAggregate using the external merge sort. Since only 1 MB of memory is available, the intermediate results cannot be kept entirely in memory. As a result, sorting and aggregation spill to disk increasing I/O cost and slowing execution.

	QUERY PLAN text
1	Finalize GroupAggregate (cost=532536.22..753192.81 rows=600122 width=40) (actual time=3917.855..13722.010 rows=5999989.00 loops=1)
2	Group Key: L_supkey, L_orderkey
3	Buffers: shared hit=15965 read=96709, temp read=50671 written=51018
4	-> Gather Merge (cost=532536.22..731288.35 rows=1440293 width=40) (actual time=3917.841..8222.074 rows=5999989.00 loops=1)
5	Workers Planned: 2
6	Workers Launched: 2
7	Buffers: shared hit=15965 read=96709, temp read=50671 written=51018
8	-> Partial GroupAggregate (cost=531536.19..564042.78 rows=600122 width=40) (actual time=3546.874..6077.260 rows=1999996.33 loops=1)
9	Group Key: L_supkey, L_orderkey
10	Buffers: shared hit=15965 read=96709, temp read=50671 written=51018
11	-> Sort (cost=531536.19..537787.46 rows=2500506 width=13) (actual time=3546.807..4065.442 rows=2000405.00 loops=3)
12	Sort Key: L_supkey, L_orderkey
13	Sort Method: external merge Disk: 48752kB
14	Buffers: shared hit=15965 read=96709, temp read=50671 written=51018
15	Worker 0: Sort Method: external merge Disk: 41952kB
16	Worker 1: Sort Method: external merge Disk: 44440kB
17	-> Parallel Seq Scan on lineitem (cost=0.00..137605.06 rows=2500506 width=13) (actual time=2.106..1516.160 rows=2000405.00 loops=1)
18	Buffers: shared hit=15892 read=96708
19	Planning:
20	Buffers: shared hit=21 read=13 dirtied=2
21	Planning Time: 8.735 ms
22	Execution Time: 14097.615 ms

- (ii) `set work_mem = '1000MB';`

With more memory, PostgreSQL can perform most operations in memory. The query plan shows that PostgreSQL uses a HashAggregate instead of GroupAggregate now and performs quicksort allowing all grouping and sorting operations to be processed in RAM without disk usage.

The HashAggregate stores the grouped results in memory hash tables keyed by (l_suppkey, l_orderkey) and computes sum(l_quantity) without relying on external storage. This greatly reduces I/O and improves performance by eliminating the need for temporary disk files.

	QUERY PLAN
	text 
1	Sort (cost=282719.19..284219.49 rows=600122 width=40) (actual time=30303.656..31372.158 rows=5999989.00 loops=1)
2	Sort Key: l_suppkey, l_orderkey
3	Sort Method: quicksort Memory: 384108kB
4	Buffers: shared hit=16099 read=96501, temp read=956 written=1687
5	-> HashAggregate (cost=217621.26..225122.79 rows=600122 width=40) (actual time=10445.467..25868.835 rows=5999989.00 l...
6	Group Key: l_suppkey, l_orderkey
7	Batches: 5 Memory Usage: 2048049kB Disk Usage: 7864kB
8	Buffers: shared hit=16099 read=96501, temp read=956 written=1687
9	-> Seq Scan on lineitem (cost=0.00..172612.15 rows=6001215 width=13) (actual time=1.502..2544.766 rows=6001215.00 loo...
10	Buffers: shared hit=16099 read=96501
11	Planning Time: 0.670 ms
12	Execution Time: 31608.306 ms

Task 6:

- (i) Explain in plain words the query plan that PostgreSQL selects to answer the query.
- (ii) Run the same query again and explain how the query plan has changed? Why did this happen?

Answer:

(i) PostgreSQL chose to use a Hash Join as there was no useful index on `l_orderkey` or `l_suppkey`. It first performed a sequential scan on `lineitem l1` to build a hash table on `l_orderkey`. Since the working memory of 1MB was insufficient for the ~7MB hash table, PostgreSQL split the hash table into 128 batches and wrote the intermediate results to disk. It then sequentially scanned `lineitem l2` and probed the hash table for matches on `l2.l_suppkey = l1.l_orderkey`.

Batches are partitions of the hash table that are written to temporary disk files, which is how PostgreSQL manages memory overflow. Each batch is then read from the disk, scanning the other table repeatedly for matches relevant to that batch. This reduces the memory usage but increases disk I/O (204068 read and writes) and CPU execution time (8338.349ms which is a significantly slower execution for a join of this size).

	QUERY PLAN text
1	Hash Join (cost=353094.02..1866736.49 rows=89708300 width=234) (actual time=1500.167..8184.088 rows=5982534.00 loops=1)
2	Hash Cond: (l2.l_suppkey = l1.l_orderkey)
3	Buffers: shared hit=32177 read=193023, temp read=204068 written=204068
4	-> Seq Scan on lineitem l2 (cost=0.00..172606.23 rows=6000623 width=117) (actual time=0.104..336.081 rows=6001215.00 loops=1)
5	Buffers: shared hit=16089 read=96511
6	-> Hash (cost=172606.23..172606.23 rows=6000623 width=117) (actual time=1478.165..1478.166 rows=6001215.00 loops=1)
7	Buckets: 65536 Batches: 128 Memory Usage: 7470kB
8	Buffers: shared hit=16088 read=96512, temp written=101927
9	-> Seq Scan on lineitem l1 (cost=0.00..172606.23 rows=6000623 width=117) (actual time=0.006..294.177 rows=6001215.00 loops=1)
10	Buffers: shared hit=16088 read=96512
11	Planning:
12	Buffers: shared hit=77 read=4 dirtied=3
13	Planning Time: 1.851 ms
14	Execution Time: 8338.349 ms

(ii) Increasing the working memory from 1MB to 64MB, PostgreSQL still chose Hash Join for the query plan. However, the new plan only has 1 batch instead of 128 and no temporary read and write operations. This is due to the fact that the entire hash table now fits in memory. Thus, no batching or disk I/O is needed, resulting in less overhead and drastically improving execution time (3215.820ms)

	QUERY PLAN text	
1	Nested Loop (cost=0.44..1380597.91 rows=89708300 width=234) (actual time=0.879..3068.603 rows=5982534.00 loops=1)	
2	Buffers: shared hit=48611 read=96754	
3	-> Seq Scan on lineitem l2 (cost=0.00..172606.23 rows=6000623 width=117) (actual time=0.129..320.350 rows=6001215.00 loops=1)	
4	Buffers: shared hit=16058 read=96542	
5	-> Memoize (cost=0.44..0.80 rows=15 width=117) (actual time=0.000..0.000 rows=1.00 loops=6001215)	
6	Cache Key: l2.l_suppkey	
7	Cache Mode: logical	
8	Hits: 5991215 Misses: 10000 Evictions: 0 Overflows: 0 Memory Usage: 2147kB	
9	Buffers: shared hit=32553 read=212	
10	-> Index Scan using lineitem_pkey on lineitem l1 (cost=0.43..0.79 rows=15 width=117) (actual time=0.001..0.001 rows=1.00 loops=10000)	
11	Index Cond: (l_orderkey = l2.l_suppkey)	
12	Index Searches: 10000	
13	Buffers: shared hit=32553 read=212	
14	Planning:	
15	Buffers: shared hit=8	
16	Planning Time: 0.207 ms	
17	Execution Time: 3215.820 ms	

Task 7:

- (i) Explain in plain words the query plan that PostgreSQL selects to answer the query. Is it different from what you saw in Task 6? Why?
- (ii) Run the same query again and explain how the query plan has changed. Why did this happen?
- (iii) Run the same query for a third time and explain how the query plan has changed. Why did this happen?

Answer:

(i) PostgreSQL selected Merge Join instead of the Hash Join used in Task 6 as the query includes an “order by l2.l_suppkey” clause. Merge Join requires both inputs to be sorted on the join keys (l2.l_suppkey = l1.l_orderkey) before merging. The query plan shows “Sort Method: external merge Disk”, meaning the sort spilled to disk due to the insufficient working memory of 1MB. Thus, PostgreSQL performed an external merge sort. The “Gather Merge” and “Workers Launched: 2” lines indicate that PostgreSQL parallelised the sequential scan of l2 to speed up reading, but disk I/O from sorting still dominated the runtime. After sorting, the two sorted streams are merged to produce the final ordered result. Due to not being able to be fit into memory, the query took a substantial amount of time to finish (4957.834 ms).

	QUERY PLAN text	
1	Merge Join (cost=1038261.28..3049787.97 rows=89708300 width=234) (actual time=2141.838..4765.200 rows=5982534.00 loops=1)	
2	Merge Cond: (l2.l_suppkey = l1.l_orderkey)	
3	Buffers: shared hit=16431 read=96743, temp read=382644 written=383634	
4	-> Gather Merge (cost=1019600.83..1718472.27 rows=6000623 width=117) (actual time=2141.807..3258.165 rows=6001215.00 loops=1)	
5	Workers Planned: 2	
6	Workers Launched: 2	
7	Buffers: shared hit=15931 read=96743, temp read=382644 written=383634	
8	-> Sort (cost=1018600.81..1024851.46 rows=2500260 width=117) (actual time=2068.488..2339.588 rows=2000405.00 loops=3)	
9	Sort Key: l2.l_suppkey	
10	Sort Method: external merge Disk: 263704kB	
11	Buffers: shared hit=15931 read=96743, temp read=382644 written=383634	
12	Worker 0: Sort Method: external merge Disk: 249256kB	
13	Worker 1: Sort Method: external merge Disk: 252360kB	
14	-> Parallel Seq Scan on lineitem l2 (cost=0.00..137602.60 rows=2500260 width=117) (actual time=0.422..125.769 rows=2000405.00 loops=3)	
15	Buffers: shared hit=15860 read=96740	
16	-> Materialize (cost=0.43..457788.55 rows=6000623 width=117) (actual time=0.025..176.507 rows=5982535.00 loops=1)	
17	Storage: Memory Maximum Storage: 18kB	
18	Buffers: shared hit=500	
19	-> Index Scan using lineitem_pkey on lineitem l1 (cost=0.43..442786.99 rows=6000623 width=117) (actual time=0.020..4.812 rows=9966.00 loops=1)	
20	Index Searches: 1	
21	Buffers: shared hit=500	
22	Planning:	
23	Buffers: shared hit=8	
24	Planning Time: 0.250 ms	
25	Execution Time: 4957.834 ms	

(ii) When the work_mem was 1MB in part (i), PostgreSQL performed an external merge sort that spilled data to the disk before executing a Merge Join. After increasing working memory to 1000MB, PostgreSQL still used Merge Join due to the “order by l2.l_suppkey” clause requiring a sorted output. However, it was able to sort the data fully in memory using the quicksort method. No temporary reads and writes were required, decreasing the overhead and execution time decreased to 4260.251ms.

This increase in working memory allowed PostgreSQL to perform in-memory operations rather than disk-based ones, reducing I/O and improving performance.

	QUERY PLAN	
	text	
1	Merge Join (cost=854431.60..2189001.98 rows=89708300 width=234) (actual time=1283.854..4034.781 rows=5982534.00 loops=1)	
2	Merge Cond: (l1.l_orderkey = l2.l_suppkey)	
3	Buffers: shared hit=16552 read=96548	
4	-> Index Scan using lineitem_pkey on lineitem l1 (cost=0.43..442786.99 rows=6000623 width=117) (actual time=0.014..5.860 rows=9966.00 loops=1)	
5	Index Searches: 1	
6	Buffers: shared hit=500	
7	-> Sort (cost=848176.80..863178.35 rows=6000623 width=117) (actual time=1283.828..2740.792 rows=10481858.00 loops=1)	
8	Sort Key: l2.l_suppkey	
9	Sort Method: quicksort Memory: 969428kB	
10	Buffers: shared hit=16052 read=96548	
11	-> Seq Scan on lineitem l2 (cost=0.00..172606.23 rows=6000623 width=117) (actual time=0.019..299.587 rows=6001215.00 loops=1)	
12	Buffers: shared hit=16052 read=96548	
13	Planning:	
14	Buffers: shared hit=8	
15	Planning Time: 0.303 ms	
16	Execution Time: 4260.251 ms	

(iii) When the working memory was increased to 2000MB, PostgreSQL again selected using Merge Join for the query due to the “order by l2.l_suppkey” clause requiring a sorted output. The sort step on l2.l_suppkey used an in-memory quicksort method which consumed 1016018kB of memory. However, since the data already fit entirely into the memory at 1000MB in part (ii), the plan and runtime remained almost unchanged (4325.761ms).

This showed that once the sort fits within the available working memory, further increasing the work_mem yields diminishing returns as PostgreSQL already avoids disk I/O.

	QUERY PLAN	
	text	🔒
1	Merge Join (cost=854431.60..2189001.98 rows=89708300 width=234) (actual time=1269.738..4126.658 rows=5982534.00 loops=1)	
2	Merge Cond: (l1.l_orderkey = l2.l_suppkey)	
3	Buffers: shared hit=16562 read=96538	
4	-> Index Scan using lineitem_pkey on lineitem l1 (cost=0.43..442786.99 rows=6000623 width=117) (actual time=0.014..6.196 rows=9966.00 loops=1)	
5	Index Searches: 1	
6	Buffers: shared hit=500	
7	-> Sort (cost=848176.80..863178.35 rows=6000623 width=117) (actual time=1269.711..2766.581 rows=10481858.00 loops=1)	
8	Sort Key: l2.l_suppkey	
9	Sort Method: quicksort Memory: 1016018kB	
10	Buffers: shared hit=16062 read=96538	
11	-> Seq Scan on lineitem l2 (cost=0.00..172606.23 rows=6000623 width=117) (actual time=0.032..302.936 rows=6001215.00 loops=1)	
12	Buffers: shared hit=16062 read=96538	
13	Planning:	
14	Buffers: shared hit=8	
15	Planning Time: 0.240 ms	
16	Execution Time: 4325.761 ms	

Task 8:

Explain in plain words the query plan that PostgreSQL selects to answer the query.
Was any sorting performed? Why did this happen?

Answer:

	QUERY PLAN text
1	Merge Join (cost=4.53..586616.84 rows=6001215 width=224) (actual time=0.574..2384.215 rows=6001215.00 loops=1)
2	Merge Cond: (l.l_orderkey = o.o_orderkey)
3	Buffers: shared hit=179325 read=165918 written=6
4	-> Index Scan using lineitem_pkey on lineitem l (cost=0.43..442795.87 rows=6001215 width=117) (actual time=0.358..1058.599 rows=6001215.00 loops=1)
5	Index Searches: 1
6	Buffers: shared hit=179074 read=135568 written=4
7	-> Index Scan using orders_pkey on orders o (cost=0.43..65100.11 rows=1500000 width=107) (actual time=0.201..270.622 rows=1500000.00 loops=1)
8	Index Searches: 1
9	Buffers: shared hit=251 read=30350 written=2
10	Planning:
11	Buffers: shared hit=55 read=28 dirtied=4
12	Planning Time: 10.401 ms
13	Execution Time: 2535.605 ms

Both inputs are already sorted on the same key of (orderkey). Hence, PostgreSQL can efficiently merge them without any additional sorting. The result of the Merge Join is automatically produced in ascending order of o_orderkey, which satisfies the ORDER BY clause directly. Therefore, no separate Sort node appears in the plan. This is the reason why no explicit sorting step was performed.

The query performs a join between the lineitem and orders tables using the condition l.l_orderkey = o.o_orderkey, and then orders the result by o_orderkey. The execution plan chosen by PostgreSQL shows that it used a Merge Join. This happens because both tables already have B-tree indexes on their primary keys: lineitem_pkey (l_orderkey, l_linenum) and orders_pkey (o_orderkey). These indexes store tuples in sorted order by the join key.

Task 9:

(i) Explain in plain words the query plan that PostgreSQL selects to answer the query. Did the plan use the index `idx_lineitem_quantity` we built? How do you explain this?

(ii) Explain in plain words the query plan that PostgreSQL selects to answer the query. Did the plan use any indexes? How do you explain this?

Answer

(i)

	QUERY PLAN text
1	Gather Merge (cost=375461.34..488044.83 rows=966660 width=224) (actual time=1510.053..1726.858 rows=898679.00 loops=1)
2	Workers Planned: 2
3	Workers Launched: 2
4	Buffers: shared hit=14403 read=124347, temp read=105658 written=107280
5	-> Sort (cost=374461.32..375468.25 rows=402775 width=224) (actual time=1490.014..1520.375 rows=299559.67 loops=3)
6	Sort Key: l.l_partkey
7	Sort Method: external merge Disk: 69752kB
8	Buffers: shared hit=14403 read=124347, temp read=105658 written=107280
9	Worker 0: Sort Method: external merge Disk: 71488kB
10	Worker 1: Sort Method: external merge Disk: 71088kB
11	-> Parallel Hash Join (cost=50574.50..208926.74 rows=402775 width=224) (actual time=896.124..1153.969 rows=299559.67 loops=3)
12	Hash Cond: (l.l_partkey = o.o_custkey)
13	Buffers: shared hit=14389 read=124347, temp read=26044 written=27420
14	-> Parallel Seq Scan on lineitem l (cost=0.00..143856.33 rows=52594 width=117) (actual time=1.447..503.653 rows=40038.00 loops=3)
15	Filter: (l_quantity = '20'::numeric)
16	Rows Removed by Filter: 1960367
17	Buffers: shared hit=12913 read=99687
18	-> Parallel Hash (cost=32386.00..32386.00 rows=625000 width=107) (actual time=344.750..344.751 rows=500000.00 loops=3)
19	Buckets: 16384 Batches: 128 Memory Usage: 1984kB
20	Buffers: shared hit=1476 read=24660, temp written=24348
21	-> Parallel Seq Scan on orders o (cost=0.00..32386.00 rows=625000 width=107) (actual time=0.507..64.467 rows=500000.00 loops=3)
22	Buffers: shared hit=1476 read=24660
23	Planning:
24	Buffers: shared hit=20 read=6
25	Planning Time: 2.055 ms
26	Execution Time: 1765.356 ms

The planner first applies the filter `l_quantity = 20`. We created an index on `l_quantity` with equality conditions and the index can help find the matching rows quickly. The usual plan shows a Bitmap Index Scan on `idx_lineitem_quantity` followed by a Bitmap Heap Scan on `lineitem` to get the actual rows.

The join `l.l_partkey = o.o_custkey` has no index in the dataset, so PostgreSQL uses a Hash Join, where one table is hashed and the other table is matched against it.

After the join, PostgreSQL sorts the result by `o_custkey` to satisfy the `ORDER BY` clause. Since `work_mem` is only 1MB, this sort might be too large to fit in memory and could spill to disk, slowing down the query.

If the plan shows “Bitmap Index Scan on `idx_lineitem_quantity`,” it means the index was used. This happens because equality on an indexed column is selective and efficient. If the plan instead shows a Sequential Scan on `lineitem`, it means the optimizer decided that many rows have `l_quantity = 20`, making a full table scan cheaper than using the index.

(ii)

It does not use `idx_lineitem_quantity`, because the predicate does not involve `l_quantity`. It may use the composite primary-key index on `lineitem` (`lineitem_pkey`) for the `l_orderkey` range. Orders’ primary-key index on `o_orderkey` is not helpful here because the join is on `o_custkey`, not `o_orderkey`, and there is no default index on `o_custkey`; therefore a Hash Join and a final Sort are expected.

Query Plan:

1. The filter is on `l_orderkey`, not on `l_quantity`, so the index `idx_lineitem_quantity` is irrelevant.
2. PostgreSQL can use the existing primary-key index on `lineitem` (`lineitem_pkey` over (`l_orderkey`, `l_linenum`)) to satisfy the range predicate `l_orderkey <= 10000`, because btree supports left-prefix range scans. Depending on selectivity and cost, you will typically see an Index Scan (or Bitmap Index Scan) on `lineitem_pkey` restricted to the `l_orderkey` range. If the range is very large, the optimizer may still choose a Sequential Scan.
3. The join `l_partkey = o_custkey` again has no supporting index by default, so a Hash Join is used.
4. `ORDER BY o_custkey` requires an explicit Sort on the join output unless there is an index on `orders(o_custkey)`. In the base schema there isn’t, so expect a Sort node; with `work_mem = 1MB` it may spill to disk.

Task 10:

- (i) What was the result size predicted by the query planner? What is the actual result size?
- (ii) How do you explain the running time differences of the above queries?
- (iii) If you can change the implementation of PostgreSQL, do you see an algorithm that can process all three queries with negligible cost differences?
- (iv) Rewrite the 3rd query into another SQL query that returns the same result in less than 30 seconds

Answer:

(i)

```
QUERY PLAN
-----
Hash Join (cost=254453.34..1532148.85 rows=92384640 width=4) (actual rows=30012985.00 loops=1)
  Hash Cond: (l1.l_orderkey = l2.l_orderkey)
  Buffers: shared hit=33 read=32813 written=7284, temp read=34676 written=34676
  -> Index Only Scan using lineitem_pkey on lineitem l1 (cost=0.43..155994.72 rows=6001215 width=4) (actual rows=6001215.00 loops=1)
    Heap Fetches: 0
    Index Searches: 1
    Buffers: shared hit=17 read=16406 written=688
  -> Hash (cost=155994.72..155994.72 rows=6001215 width=4) (actual rows=6001215.00 loops=1)
    Buckets: 262144 Batches: 64 Memory Usage: 5372kB
    Buffers: shared hit=16 read=16407 written=6596, temp written=17275
    -> Index Only Scan using lineitem_pkey on lineitem l2 (cost=0.43..155994.72 rows=6001215 width=4) (actual rows=6001215.00 loops=1)
      Heap Fetches: 0
      Index Searches: 1
      Buffers: shared hit=16 read=16407 written=6596
Planning:
  Buffers: shared hit=170 read=7
Planning Time: 3.069 ms
Execution Time: 3314.227 ms
(18 rows)
```

Predicted: 92,384,640 rows

Actual: 30,012,985 rows

Execution Time: ~3.31 s

(ii)

```
SELECT COUNT(*) AS lineitem_rows FROM lineitem;
SQL

psql -p 5433 -d proj2 -f ~/tpch-lab/task10_q2_pg18_part1.sql

Pager usage is off.
Timing is on.
  lineitem_rows
-----
          6001215
(1 row)

Time: 244.262 ms
```

Time: ~0.24 s

Why fast: one scan of lineitem with a simple aggregate; no joins, no spills.

```
SELECT COUNT(*) AS pairs_naive
FROM lineitem l1
JOIN lineitem l2 ON l1.l_orderkey = l2.l_orderkey;
SQL

psql -p 5433 -d proj2 -f ~/tpch-lab/task10_q2_pg18_part2_count.sql

Pager usage is off.
Timing is on.
  pairs_naive
-----
      30012985
(1 row)

Time: 1311.282 ms (00:01.311)
```

Time: ~1.31 s.

Why slower: reads table twice and spills hash to disk → extra passes.

```
psql -p 5433 -d proj2 -f ~/tpch-lab/task10_q2_pg18_part3_naive.sql

Pager usage is off.
Timing is on.
SET
Time: 0.743 ms
  triples_naive
-----
      168109581
(1 row)

Time: 5546.768 ms (00:05.547)
RESET
Time: 2.025 ms
```

Time: ~5.55 s.

Why slowest: combinatorial blow-up per orderkey ($\approx \sum n^3$) + big hashes + temp I/O.

(iii)

```
WITH cnt AS (  
  SELECT l_orderkey, COUNT(*)::bigint AS n  
  FROM lineitem  
  GROUP BY l_orderkey  
)  
SELECT  
  SUM(n)      AS lineitem_rows,  
  SUM(n*n)    AS pairs_count,  
  SUM(n*n*n)  AS triples_count  
FROM cnt;  
SQL
```

```
psql -p 5433 -d proj2 -f ~/tpch-lab/task10_q3_pg18_oneshot.sql
```

Pager usage is off.
Timing is on.

lineitem_rows	pairs_count	triples_count
6001215	30012985	168109581

(1 row)

Time: 542.061 ms

Explain the optimizer rewrite: pre-aggregate ***COUNT(*) BY l_orderkey*** once (n per key), then use closed-form sums ($\sum n$, $\sum n^2$, $\sum n^3$). That makes all three tasks a single pass + tiny aggregation → similar runtimes.

(iv)

```
WITH cnt AS (  
  SELECT l_orderkey, COUNT(*)::bigint AS n  
  FROM lineitem  
  GROUP BY l_orderkey  
)  
SELECT SUM(n*n*n) AS triples_fast FROM cnt;  
SQL
```

```
psql -p 5433 -d proj2 -f ~/tpch-lab/task10_q2_pg18_part3_fast.sql
```

Pager usage is off.
Timing is on.

triples_fast
168109581

(1 row)

Time: 607.732 ms

triples_naive: ~5.55 s

triples_fast: ~0.61 s

Transactions

Task 11:

- (i) Are the results of Q1 and Q2 identical?
- (ii) What anomalies — according to the lecture slides “Transactions 2: Isolation Levels” — have you witnessed?
- (iii) Why is this allowed?

Answer:

- (i) They are not identical. One only has a row of data while the other has 2 rows.
- (ii) This is a phantoms anomaly.
- (iii) This is allowed because PostgreSQL default isolation level is read-committed, which only avoids dirty-read anomalies. If this is to be prevented, it requires predicate locking or serialization, which is very costly in normal database usage. It also reduces the throughput of data.

```
SC3020 Project 2=# insert into public.t values(1, 10);
INSERT 0 1
SC3020 Project 2=# START TRANSACTION ISOLATION LEVEL READ COMMITTED;
START TRANSACTION
SC3020 Project 2=# select * from public.t;
 id | v
----+----
  1 | 10
(1 row)

SC3020 Project 2=# select * from public.t;
 id | v
----+----
  1 | 10
  2 | 20
(2 rows)
```

```
SC3020 Project 2=# START TRANSACTION ISOLATION LEVEL SERIALIZABLE;
START TRANSACTION
SC3020 Project 2=# insert into public.t values(2, 20);
INSERT 0 1
SC3020 Project 2=# COMMIT;
COMMIT
```

Task 12:

What do you observe? Why does this happen?

Setup (once):

```
proj2=# DROP TABLE IF EXISTS public.t;  
CREATE TABLE public.t(id int PRIMARY KEY, val int NOT NULL);  
CREATE INDEX ON public.t(val);  
INSERT INTO public.t VALUES (1,10);  
DROP TABLE  
CREATE TABLE  
CREATE INDEX  
INSERT 0 1
```

Session A:

```
proj2=# BEGIN ISOLATION LEVEL SERIALIZABLE;  
SELECT count(*) FROM public.t WHERE val=100;  
INSERT INTO public.t VALUES (3,100);  
BEGIN  
count  
-----  
0  
(1 row)  
  
INSERT 0 1
```

Session B:

```
proj2=# BEGIN ISOLATION LEVEL SERIALIZABLE;  
SELECT count(*) FROM public.t WHERE val=100;  
INSERT INTO public.t VALUES (2,100);  
BEGIN  
count  
-----  
0  
(1 row)  
  
INSERT 0 1
```

Session A:

```
proj2=*# COMMIT;  
COMMIT
```


Session B:

```
proj2==# COMMIT;  
ERROR:  could not serialize access due to read/write dependencies among transactions  
DETAIL:  Reason code: Canceled on identification as a pivot, during commit attempt.  
HINT:   The transaction might succeed if retried.
```

Final state:

```
proj2=# SELECT * FROM public.t ORDER BY id;  
 id | val  
----+-----  
  1 |   10  
  3 |  100  
(2 rows)
```

Answers:

What we observed:

- Both transactions read “no rows with *val* = 100”.
- Each then inserted a row with *val* = 100.
- PostgreSQL aborted one transaction at commit with a serialization failure.
- The other transaction committed; only one *val* = 100 row remains.

Why this happens:

- Under SERIALIZABLE, PostgreSQL’s **Serializable Snapshot Isolation (SSI)** tracks predicate reads (range reads like WHERE val = 100) using SIREAD locks (the index on *val* lets SSI lock the searched value/range precisely).
- Each transaction read “no 100s” and then wrote a row that would have been visible to the other’s read. That creates a dangerous rw-dependency cycle (classic write-skew/phantom pattern).
- SSI must break the cycle; it chooses a pivot transaction and aborts it at **COMMIT** (the error text you saw).
- Result: the outcome is equivalent to some serial order (only one insert “wins”).

In other words, the Serialization isolation level ensures that all transactions behave as if they are executed one after another. T1 started before T2, seeing the table before the row (2,20) exists. Then T2 inserts and commits (2,20) changing the table’s state. When T1 later tries to insert (3,30), PostgreSQL detects the data it relied on is no longer consistent. The data T1 used is outdated and conflicts with T2’s changes. Hence, PostgreSQL aborts T1 with a serialization error to maintain correctness.

Task 13:

What do you observe? Why does this happen?

T1 Session:

```
proj2=# delete from public.T where id >= 2;
```

```
DELETE 0
```

```
proj2=# insert into public.t values(1, 10);
```

```
ERROR: duplicate key value violates unique constraint "t_pkey"
```

```
DETAIL: Key (id)=(1) already exists.
```

```
proj2=# START TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

```
START TRANSACTION
```

```
proj2=*# update public.T set v = 20 where id = 1;
```

```
UPDATE 1
```

T2 Session:

```
proj2=# START TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

```
START TRANSACTION
```

```
proj2=*# update public.T set v = 30 where id = 1;
```

Answer:

What we observed : In the T2 session, it stops processing the query. The output will be (1,20).

Why this happens : This is because the T1 session has held the row lock, which prevents the T2 session from running until T1 has been committed.

After COMMIT:

What do you observe? Why does this happen?

T1 Session:

```
proj2=# delete from public.T where id >= 2;
```

```
DELETE 0
```

```
proj2=# insert into public.t values(1, 10);
```

```
ERROR: duplicate key value violates unique constraint "t_pkey"
```

```
DETAIL: Key (id)=(1) already exists.
```

```
proj2=# START TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

```
START TRANSACTION
```

```
proj2=*# update public.T set v = 20 where id = 1;
```

```
UPDATE 1
```

```
proj2=*# COMMIT;
```

```
COMMIT
```

T2 Session:

```
proj2=# START TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

```
START TRANSACTION
```

```
proj2=*# update public.T set v = 30 where id = 1;
```

```
UPDATE 1
```

Answer:

What we observed : The moment the commit is done, the T2 session starts running. The value has been updated to (1,30) instead.

Why this happens : This is because the T1 session has been committed. This allows T2 to wake up and re-read the row at **READ COMMITTED** visibility.

It applies its own updates and sets the v value from 20 to 30.

T2 Session:

```
proj2=# select * from public.t;
 id | v
----+---
  1 | 30
(1 row)
```

Answer:

What we observed: The T2 session displays (1, 30) because the session can always see its own uncommitted writes.

T2 Session:

```
proj2=# ROLLBACK;
ROLLBACK
proj2=# select * from public.t;
 id | v
----+---
  1 | 20
(1 row)
```

Answer:

What we observed: The value reverts to (1, 20), which is the value committed by T1. The rollback discarded T2's change.

Why this happens: The ROLLBACK command reverts all uncommitted changes made in that transaction. This causes PostgreSQL to discard T2's update (1, 30) and restore the last committed value (1, 20) by T1.